

Formalization Seminar: Kickoff

George McNinch

Tufts University

2026-09-15

Welcome	1
Plan for today	2
Proof assistants and Lean	3
Live demo	8
Where this is headed	10
Logistics	14
Questions	15

- proof assistant and *Lean*?

Plan for today

Welcome

- proof assistant and *Lean*?
- a short live demo – watch Lean check a proof

- proof assistant and *Lean*?
- a short live demo – watch Lean check a proof
- where I'd like to head: formalizing some undergraduate analysis

- proof assistant and *Lean*?
- a short live demo – watch Lean check a proof
- where I'd like to head: formalizing some undergraduate analysis
- logistics

Welcome	1
Proof assistants and Lean	3
What is a proof assistant?	4
Why Lean, specifically?	5
A taste of what Lean looks like	6
Live demo	8
Where this is headed	10
Logistics	14
Questions	15

What is a proof assistant?

Proof assistants and Lean

What is a proof assistant?

- It is a piece of software for:
 - **defining** mathematical objects,
 - **specifying** properties of those objects,
 - **proving** that the specifications hold

What is a proof assistant?

- It is a piece of software for:
 - **defining** mathematical objects,
 - **specifying** properties of those objects,
 - **proving** that the specifications hold
- the system (“proof assistant”) checks that the proof is correct. This check feels a bit like a *compilation step*.

What is a proof assistant?

- It is a piece of software for:
 - **defining** mathematical objects,
 - **specifying** properties of those objects,
 - **proving** that the specifications hold
- the system (“proof assistant”) checks that the proof is correct. This check feels a bit like a *compilation step*.
- contrast with automated theorem proving: there, the **computer** searches for the argument. Here, **you** construct it and the proof assistant checks it.

What is a proof assistant?

- It is a piece of software for:
 - **defining** mathematical objects,
 - **specifying** properties of those objects,
 - **proving** that the specifications hold
- the system (“proof assistant”) checks that the proof is correct. This check feels a bit like a *compilation step*.
- contrast with automated theorem proving: there, the **computer** searches for the argument. Here, **you** construct it and the proof assistant checks it.
- there are many different examples of proof assistants: Lean, Coq, Isabelle/HOL, Agda, Mizar, ...

The differences mostly come down to the choice of underlying logic.

Why Lean, specifically?

Proof assistants and Lean

Why Lean, specifically?

- Lean's foundations – **dependent type theory** (Calculus of Constructions with inductive types) – is expressive enough to state and prove essentially any theorem you'd meet in ordinary mathematics

Why Lean, specifically?

- Lean's foundations – **dependent type theory** (Calculus of Constructions with inductive types) – is expressive enough to state and prove essentially any theorem you'd meet in ordinary mathematics
- There is a large, actively maintained, community library of formalized mathematics known as **mathlib** – see <https://github.com/leanprover-community/mathlib4>

Why Lean, specifically?

- Lean's foundations – **dependent type theory** (Calculus of Constructions with inductive types) – is expressive enough to state and prove essentially any theorem you'd meet in ordinary mathematics
- There is a large, actively maintained, community library of formalized mathematics known as **mathlib** – see <https://github.com/leanprover-community/mathlib4>
- Mainly, this seminar will take a pragmatic view on this subject. Rather than describing type-theoretic foundations for their own sake – we'll just focus on how to use Lean to formalize mathematics.


```
-- a definition
def append {α : Type} (xs ys : List α) : List α :=
  match xs with
  | []      => ys
  | z :: zs => z :: append zs ys
```



```
-- a definition
def append {α : Type} (xs ys : List α) : List α :=
  match xs with
  | []      => ys
  | z :: zs => z :: append zs ys

-- a theorem, and its proof
theorem append_length {α : Type} (xs ys : List α) :
  (append xs ys).length = xs.length + ys.length := by
  induction xs with
  | nil => simp [append]
  | cons z zs ih =>
    simp only [append, List.length_cons]
    rw [ih]
    ring
```



```
-- a definition
def append {α : Type} (xs ys : List α) : List α :=
  match xs with
  | []      => ys
  | z :: zs => z :: append zs ys

-- a theorem, and its proof
theorem append_length {α : Type} (xs ys : List α) :
  (append xs ys).length = xs.length + ys.length := by
  induction xs with
  | nil => simp [append]
  | cons z zs ih =>
    simp only [append, List.length_cons]
    rw [ih]
    ring
```

- this example is small, but it already has a lot: a definition, a statement, and an inductive proof.

Welcome	1
Proof assistants and Lean	3
Live demo	8
Let's watch Lean check a proof	9
Where this is headed	10
Logistics	14
Questions	15

Let's watch Lean check a proof

Live demo

- let's switch to the editor now
- we'll prove `append_length` from scratch, live
- watch for what happens if a step is **wrong**?

Welcome	1
Proof assistants and Lean	3
Live demo	8
Where this is headed	10
The target: formalizing undergraduate analysis	11
Suggested reading	13
Logistics	14
Questions	15

The target: formalizing undergraduate analysis

Where this is headed

- the plan for this seminar: work, together, toward formalizing pieces of the undergraduate analysis course

The target: formalizing undergraduate analysis

Where this is headed

- the plan for this seminar: work, together, toward formalizing pieces of the undergraduate analysis course
- here's the kind of statement we mean – convergence of a sequence:

-- "u tends to ℓ "

```
def seq_limit (u :  $\mathbb{N} \rightarrow \mathbb{R}$ ) ( $\ell$  :  $\mathbb{R}$ ) :=  
   $\forall \varepsilon > 0, \exists N, \forall n \geq N, |u\ n - \ell| \leq \varepsilon$ 
```


The target: formalizing undergraduate analysis

Where this is headed

- the plan for this seminar: work, together, toward formalizing pieces of the undergraduate analysis course
- here's the kind of statement we mean – convergence of a sequence:

-- "u tends to ℓ "

```
def seq_limit (u :  $\mathbb{N} \rightarrow \mathbb{R}$ ) ( $\ell$  :  $\mathbb{R}$ ) :=  
   $\forall \varepsilon > 0, \exists N, \forall n \geq N, |u\ n - \ell| \leq \varepsilon$ 
```

- and a real theorem about it – the squeeze theorem:

```
theorem squeeze (hu : seq_limit u  $\ell$ ) (hw : seq_limit w  $\ell$ )  
  (h :  $\forall n, u\ n \leq v\ n$ ) (h' :  $\forall n, v\ n \leq w\ n$ ) :  
  seq_limit v  $\ell$  := by  
sorry
```


The target: formalizing undergraduate analysis

Where this is headed

- the plan for this seminar: work, together, toward formalizing pieces of the undergraduate analysis course
- here's the kind of statement we mean – convergence of a sequence:

-- "u tends to ℓ "

```
def seq_limit (u :  $\mathbb{N} \rightarrow \mathbb{R}$ ) ( $\ell$  :  $\mathbb{R}$ ) :=  
   $\forall \varepsilon > 0, \exists N, \forall n \geq N, |u\ n - \ell| \leq \varepsilon$ 
```

- and a real theorem about it – the squeeze theorem:

```
theorem squeeze (hu : seq_limit u  $\ell$ ) (hw : seq_limit w  $\ell$ )  
  (h :  $\forall n, u\ n \leq v\ n$ ) (h' :  $\forall n, v\ n \leq w\ n$ ) :  
  seq_limit v  $\ell$  := by  
sorry
```

The target: formalizing undergraduate analysis

Where this is headed

- **that** sorry is where the semester's work lives – formalizing proofs you already know, and discovering exactly what they demand once every step must be made fully explicit

Suggested reading

Where this is headed

- a gentle, no-install warm-up: the Natural Number Game – builds tactic and induction intuition in the browser

Suggested reading

Where this is headed

- a gentle, no-install warm-up: the Natural Number Game – builds tactic and induction intuition in the browser
- when you're ready to install: Lean + mathlib (link already sent – VS Code, then the Lean extension, then mathlib)

- a gentle, no-install warm-up: the Natural Number Game – builds tactic and induction intuition in the browser
- when you're ready to install: Lean + mathlib (link already sent – VS Code, then the Lean extension, then mathlib)
- from *Mathematics in Lean*: §2.3 (“Using Theorems and Lemmas”) is the most useful part of Ch. 2 for where we're headed – the rest of Ch. 2 you can skim or skip. Then Ch. 3 (Logic), through §3.6 (“Sequences and Convergence”). Then Ch. 4 (Sets and Functions).

- a gentle, no-install warm-up: the Natural Number Game – builds tactic and induction intuition in the browser
- when you're ready to install: Lean + mathlib (link already sent – VS Code, then the Lean extension, then mathlib)
- from *Mathematics in Lean*: §2.3 (“Using Theorems and Lemmas”) is the most useful part of Ch. 2 for where we're headed – the rest of Ch. 2 you can skim or skip. Then Ch. 3 (Logic), through §3.6 (“Sequences and Convergence”). Then Ch. 4 (Sets and Functions).
- try some of the exercises as you go – no need to do all of them

Suggested reading

- a gentle, no-install warm-up: the Natural Number Game – builds tactic and induction intuition in the browser
- when you're ready to install: Lean + mathlib (link already sent – VS Code, then the Lean extension, then mathlib)
- from *Mathematics in Lean*: §2.3 (“Using Theorems and Lemmas”) is the most useful part of Ch. 2 for where we're headed – the rest of Ch. 2 you can skim or skip. Then Ch. 3 (Logic), through §3.6 (“Sequences and Convergence”). Then Ch. 4 (Sets and Functions).
- try some of the exercises as you go – no need to do all of them
- happy to help outside of meeting time too

Welcome	1
Proof assistants and Lean	3
Live demo	8
Where this is headed	10
Logistics	14
Questions	15

- weekly, Tuesdays 3:30 – 4:20, JCC 302
- see the seminar page: [gmcninch.math.tufts.edu/
pages/2026-Fall---lean-seminar.html](http://gmcninch.math.tufts.edu/pages/2026-Fall---lean-seminar.html)
- some write-ups and materials will be posted to Proof Sketches gmcninch.math.tufts.edu/proof-sketches/ as we go

Welcome	1
Proof assistants and Lean	3
Live demo	8
Where this is headed	10
Logistics	14
Questions	15
Questions?	16

- what would make this useful for **you**?